

Table of content

Table of content	1
1. Architecture Design	2
Justification	2
Main Flow:	2
UML design:	2
2. Design Pattern	3
Repository Pattern	3
3. Authentication and Authorization	3
4. Database Design	3
Tables & Relationships:	3
Database Schema	4
Assumptions:	4
5- Business Logic	5
6- Error Handling	5

1. Architecture Design

The application follows a **layered architecture** within a single Web API project. The application is also **organized by feature**, allowing related components such as controllers, services, DTOs, and business logic to remain grouped together.

Each layer has a clearly defined responsibility, which improves maintainability and testability while supporting the **Single Responsibility Principle (SRP)**.

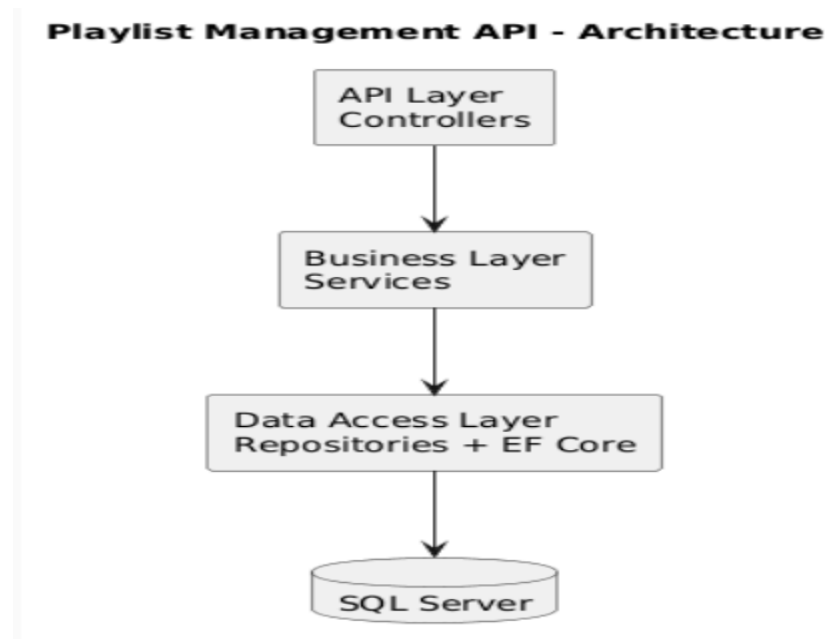
Justification

I decided to select this architecture over a multi-project Clean Architecture because the application is relatively small in scope and does not require the additional complexity of multiple projects.

Main Flow:

Controller → Service → Repository → Entity Framework Core → SQL Server

UML design:



2. Design Pattern

Repository Pattern

to isolate database-access logic from the business layer and to apply with DI principle.

3. Authentication and Authorization

The API uses JWT-based authentication.

4. Database Design

SQL Server was selected as the database because the application's domain is strongly relational.

A relational database was therefore considered more appropriate than a NoSQL database for this domain because the relationships between users, playlists, and songs are central to the business model.

Tables & Relationships:

User : stores application users.

Playlist : represents a playlist created by a user

Song : represents a song in the application's song catalog

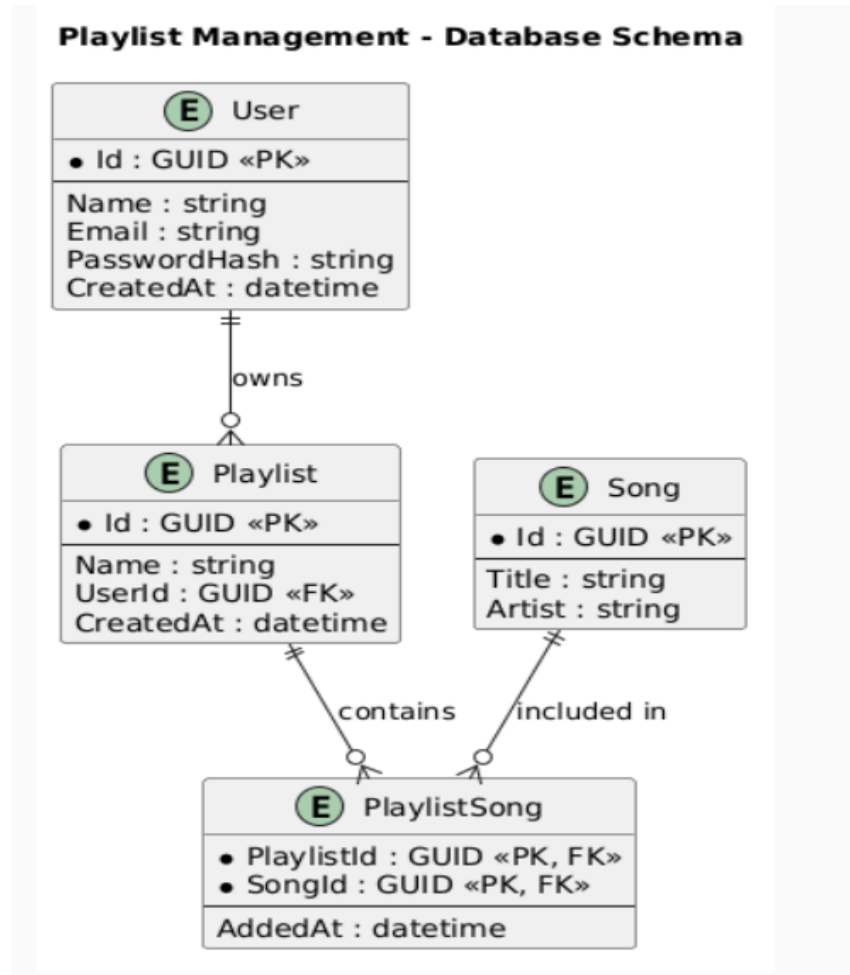
PlaylistSong: represents the conjunction table due to the M:M relationship

The main relationships are:

- A **User** can have multiple **Playlists**.
- A **Playlist** belongs to one **User**.
- A **Playlist** can contain multiple **Songs**.
- A **Song** can belong to multiple **Playlists**.

```
User 1 ————— N Playlist
Playlist 1 ————— N PlaylistSong N ————— 1 Song
```

Database Schema



Assumptions:

Songs are considered part of an **existing song catalog** within the scope of this assessment, so initial song data is provided through database seeding.

Song management is intentionally outside the current scope, as the requirements focus on creating playlists, adding songs to playlists, and retrieving playlists.

As a future enhancement, **role-based authorization** could be introduced to allow users with an **Admin** role to manage the song catalog through dedicated Song CRUD operations.

5- Business Logic

- Users can create playlists and each playlist is associated with its owner.
- Users can add existing songs to their own playlists.
-
- A song cannot be added more than once to the same playlist.
- Users can retrieve only their own playlists.
- Users can update and delete only playlists they own.
- Users can remove songs only from their own playlists.
- The system validates that the required playlist and song exist before performing operations.
- Authentication is required for playlist operations, and ownership checks are applied to protect user resources.
- Appropriate HTTP responses are returned for common cases such as not found, unauthorized, forbidden, and conflict.
- Playlist deletion uses hard delete, as recovery and auditing are outside the scope of this application.

6- Error Handling

The application uses custom exceptions for expected business failures, including:

- NotFoundException
- ForbiddenException
- ConflictException

A global exception-handling middleware converts these exceptions into appropriate HTTP responses.